

MATLAB Audio-to-Image Pipeline

File: hyo_audio_to_img_esc_20240909.mlx — Step-by-Step Explanation

Overview

This MATLAB Live Script (.mlx) implements a complete preprocessing pipeline that converts raw ESC-50 audio files into 2D scalogram images for use in deep learning models. It is the foundation of the audio-to-image classification approach used in the PhD research — turning sound classification into an image classification problem that CNNs and Transformers can process.

The Big Picture

```
ESC-50 .wav files
|
v
Mono + 16-bit + 44.1kHz normalisation
|
v
Continuous Wavelet Transform (CWT)
|
v
2D Scalogram (time x frequency)
|
v
224 x 224 colour JPG image
|
v
Organised by class label
|
v
Ready for CNN / Transformer training
```

Step 1: Load the Audio Dataset

The script begins by creating a MATLAB `audioDatastore` object pointing to the ESC-50 dataset folder containing 16kHz .wav files. This manages large collections of audio files efficiently without loading them all into memory at once. Labels are extracted from filenames rather than folder names, as ESC-50 encodes the class in its filename format.

```
ads = audioDatastore(datafolder, ...
    'IncludeSubfolders', true, ...
    'FileExtensions', '.wav', ...
    'LabelSource', 'none');
```

Step 2: Extract Label Information

The ESC-50 filename format encodes the class label within the filename itself (e.g. 1-100032-A-0.wav where 0 is the class index). The script splits the file path and filename using

delimiters to extract the label for each audio file.

```
file_dir = split(ads.Files, "/")
file_name = file_dir(:, 8)
file_name_split = split(file_name, ["-", "."])
```

Step 3: Create Output Folder Structure

Before saving images, the script creates one subfolder per class label inside the output images directory. This organises the images by class, making them easy to load into deep learning frameworks like MATLAB's imageDatastore or Python's ImageFolder.

```
for i = 1:length(catego)
    imgLoc = fullfile(datafolder_scallogram, char(catego{i}));
    mkdir(imgLoc)
end
```

Step 4: Audio Preprocessing & Normalisation

Three normalisation steps are applied to every audio file to ensure consistency across the dataset before feature extraction:

- **Stereo to Mono** — averages left and right channels into a single channel
- **Sample Rate** — standardised to 44,100 Hz for all files
- **Bit Depth** — standardised to 16-bit resolution

```
for i = 1:length(ads.Files)
    y = audioread(ads.Files{i});
    if size(y, 2) == 1
        result = y; % already mono
    else
        result = (y(:,1) + y(:,2)) / 2; % stereo to mono
    end
    audiowrite(ads.Files{i}, result, 44100, 'BitsPerSample', 16);
end
```

Step 5: Convert Audio to Scalogram Image

This is the core transformation of the pipeline. Each audio file is converted into a 2D scalogram image using the Continuous Wavelet Transform (CWT):

- **CWT Filter Bank** — analyses the signal across all frequencies and time simultaneously, with 4 voices per octave for frequency resolution
- **Scalogram** — the magnitude of wavelet coefficients forms a 2D time-frequency representation
- **Colour Mapping** — the scalogram is rescaled, converted to uint8, and mapped to RGB using the jet colourmap (128 colours)
- **Resize to 224x224** — the standard input size for CNN architectures such as ResNet, VGG, AlexNet, and Vision Transformers

```
fb = cwtfilterbank('SignalLength', length(sig), ...
```

```
'SamplingFrequency', Fs, ...  
'VoicesPerOctave', 4);  
  
cfs = abs(fb.wt(sig(:))); % Compute scalogram  
im = ind2rgb(im2uint8(rescale(cfs)), jet(128)); % Map to colour  
imwrite(imresize(im, [224 224]), fullfile(imgLoc, imFileName));
```

Step 6: Save Images Organised by Label

Each image is saved as a .jpg file inside the subfolder corresponding to its class label. The filename preserves the original audio filename for traceability. This folder structure is directly compatible with deep learning image datastores in both MATLAB and Python (PyTorch ImageFolder).

Why This Approach?

Converting audio to scalogram images allows powerful image-based deep learning architectures — CNNs and Vision Transformers — to be directly applied to audio classification tasks. The scalogram captures both time and frequency information simultaneously, making it a richer representation than a simple waveform or mel spectrogram. Resizing to 224x224 ensures compatibility with pretrained ImageNet models, enabling transfer learning from visual domains to audio.